

Problem 1: Self assessment.

For problem 7 part b). I think my solution was better since it actually solved the geometric distribution and did not just state it.

$$\begin{aligned}\mathbb{E}[R] &= \sum_{i=1}^k \mathbb{E}[R_i] = k\mathbb{E}[R_1] \\ &= k \sum_{i=1}^{\infty} i \mathbb{P}[R_1 = i] \\ &\leq k \sum_{i=1}^{\infty} i (1-p)^{i-1} p \\ &= \frac{kp}{(1 - 1 + p)^2} = \frac{k}{p}\end{aligned}$$

Problem 2: Short-answer potpourri.

(a)

Suppose that Alice's bit string is $x = 101111000$ and Bob's bit string is $y = 111100110$. State, with justification, **all** the "bad" primes p for which the fingerprinting protocol from class will (wrongly) *accept*, even though $x \neq y$.

Proof.

$x = 376, y = 486$. Since $486 - 376 = 110 = 2 \times 5 \times 11$. If $p \mid 110$. Then this implies that $p \mid (y - x) \implies x \equiv y \pmod{p}$. Thus, the only bad primes are 2, 5, 11. ■

(b)

Recall the fast "spot checking" algorithm for verifying matrix multiplication from lecture. Consider the matrices

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 3 & 2 & 1 \\ 2 & 1 & 3 \end{bmatrix}, \quad B = \begin{bmatrix} 3 & 2 & 4 \\ 1 & 3 & 3 \\ 2 & 5 & 1 \end{bmatrix}, \quad A \cdot B = \begin{bmatrix} 11 & 23 & 13 \\ 13 & 17 & 19 \\ 13 & 22 & 14 \end{bmatrix}, \quad C = \begin{bmatrix} 13 & 21 & 18 \\ 13 & 17 & 19 \\ 14 & 21 & 10 \end{bmatrix},$$

and notice that $AB \neq C$. Give, with justification, **all** the binary vectors for which the algorithm (wrongly) accepts.

Proof.

The algorithm will accept if $A \times (Br) = Cr$. Thus, this is equivalent to $(C - AB)r = 0$. Thus,

$$C - AB = \begin{bmatrix} 2 & -2 & 5 \\ 0 & 0 & 0 \\ 1 & -1 & -4 \end{bmatrix}$$

Thus, for $(C - AB)r = 0$ we get the equations.

$$2x - 2y + 5z = 0 \quad 1x - y - 4z = 0$$

Thus, we get when adding -2 times equation 2 to 1, $13z = 0 \implies z = 0$. Then when plugging this in we get $x = y$.

Thus, all the binary vectors are

$$r = \begin{bmatrix} x \\ x \\ 0 \end{bmatrix}$$

For $x \in \{0,1\}$

(c)

The ciphertext is:

400644095036345936514356093209515259423656091750325118

Recover the original message. You may use a computer or the ASCII code to string converter. Include an explanation of what you did and clearly state the secret key.

Proof.

The way I solved it was brute force checking subtracting $k \in \{0,1,\dots,9\} \pmod{10}$. Then converting to ASCII and seeing what the message was.

The message was I'M SECRETLY A TURKEY (SAT) with secret key $k = 7$.

(d)

Show that if $\gcd(x,y) = 1$ (in other words, x and y are coprime), then $ax \equiv 1 \pmod{y}$ and $by \equiv 1 \pmod{x}$. Then, calculate $32^{-1} \pmod{263}$ and $263^{-1} \pmod{32}$ using EXTENDED EUCLID. Show your work by filling in the table below:

Proof.

Assume $\gcd(x,y) = 1$. Then by Bezout's identity there is an $a,b \in \mathbb{Z}$ s.t. $ax + by = 1$. This implies that $ax = b(-y) + 1 \implies ax \equiv 1 \pmod{y}$, this is the same for by .

input		division		rec ans		output	
x	y	q	r	a'	b'	a	b
263	32	8	7	2	-9	-9	74
32	7	4	4	-1	2	2	-9
7	4	1	3	1	-1	-1	2
4	3	1	1	0	1	1	-1
3	1	3	0	1	0	0	1
1	0						

Thus, $32^{-1} \equiv 74 \pmod{263}$ and $263^{-1} \equiv -9 \equiv 23 \pmod{32}$

(e)

Compute $6^{25} \bmod 22$ using REPEATSQUARE. Show your work by first filling out the table P below (using lines 3-6). Then, list all the $P[i]$'s that you included in your product to calculate $6^{25} \bmod 22$ (using lines 8-10) in your calculation leading to the final answer.

Proof.

i	2^i	$P[i]$
0	1	6
1	2	14
2	4	20
3	8	4
4	16	16

The $P[i]$'s used were when $i \in \{0,3,4\}$

So, $6^{25} \equiv 10 \pmod{22}$. ■

Problem 3: Trading space for time in skip lists.

Recall that a skip list is essentially a linked list with multiple levels. To add an item, we first find the appropriate place to insert it in the first level, and do so. Then, we run the following loop: flip a fair coin; if it comes up heads, insert the item at the next-higher level and repeat; if it comes up tails, stop.

A skip list keeps duplicate copies of items, which consumes extra memory. In this problem, we will obtain a trade-off between the amount of memory used and the running time of the ‘find’ operation, by adjusting the probability with which we “promote” items to higher levels. That is, instead of using a fair coin, we use one that comes up heads with some probability $p \in (0,1)$.

Throughout this problem, assume that there are n distinct items in the data structure.

(a)

For any particular item in the data structure, derive the expected number of copies of it that appear in the structure, in terms of p .

Proof.

The total number of copies depends on the highest level one item gets to. Thus, let X_i be 1 if our item is on level i , and zero otherwise.

Thus, $X = \sum_{i=0}^{\infty} X_i$ if X is the number of copies of the item.

Thus, $\mathbb{E}[X] = \sum_{i=0}^{\infty} \mathbb{E}[X_i] = \sum_{i=0}^{\infty} \mathbb{P}[X_i = 1] = \sum_{i=0}^{\infty} p^i = \frac{1}{1-p}$. ■

(b)

Derive the expected size of the structure, i.e., the total number of copies of all the items, in terms of n and p . Also briefly describe how this behaves as p decreases and approaches 0.

Proof.

Let X be the total number of elements. Let X_i be the number of copies of item i .

Thus, $X = X_1 + \cdots + X_n$.

$\mathbb{E}[X] = \sum_{i=1}^n \mathbb{E}[X_i]$. From part (a) we know the expected number of copies for any one item. Therefore,

$$\mathbb{E}[X] = \sum_{i=1}^n \frac{1}{1-p} = \frac{n}{1-p}$$

As p approaches 0 $\mathbb{E}[X] = n/(1-p) \rightarrow n$. ■

(c)

Derive the expected number of items on level i (where $i = 0$ corresponds to the first level), in terms of n , p , and i .

Proof.

Let X be the total number of elements on level i . Let X_j be one if item j is on level i , and zero otherwise.

Thus, $\mathbb{E}[X_j] = \mathbb{P}[X_j = 1] = p^i$.

Therefore, since $X = X_1 + \dots + X_n \implies \mathbb{E}[X] = \mathbb{E}[X_1] + \dots + \mathbb{E}[X_n] = np^i$. ■

(d)

Let Z be random variable denoting the total number of occupied levels in the structure. Show that

$$\mathbb{E}[Z] \leq \frac{1}{1-p} + \frac{\log n}{\log(1/p)}.$$

Also briefly describe how this bound behaves as p decreases and approaches 0.

Proof.

Let X_i be the number of elements on level i . From part (c) we know that $\mathbb{E}[X_i] = np^i$

Let Z_i be one if there is at least one item on level i , and zero otherwise.

Thus, $Z = \sum_{i=0}^{\infty} Z_i$.

Therefore,

$$\begin{aligned} \mathbb{E}[Z] &= \sum_{i=0}^{\infty} \mathbb{E}[Z_i] \\ &= \sum_{i=0}^{\infty} \mathbb{P}[X_i \geq 1] \end{aligned}$$

For $i \leq \lfloor \log_{1/p} n \rfloor$, we get that $\mathbb{P}(X_i \geq 1) \leq np^i \leq np^{\log_{1/p} n} = n \times 1/n = 1$

For $i > \lfloor \log_{1/p} n \rfloor$, we get that $\mathbb{P}(X_i \geq 1) \leq np^i$ so,

$$\begin{aligned} &\leq \lfloor \log_{1/p} n \rfloor + \sum_{\lfloor \log_{1/p} n \rfloor + 1}^{\infty} np^i \\ &= \log_{1/p} n + \frac{np^{\lfloor \log_{1/p} n \rfloor + 1}}{1 - p} \end{aligned}$$

Since $p^{\lfloor \log_{1/p} n \rfloor + 1} \leq p^{\log_{1/p} n} = 1/n$

$$\leq \log_{1/p} n + \frac{1}{1 - p} = \frac{1}{1 - p} + \frac{\log n}{\log(1/p)}$$

As $p \rightarrow 0$, we can see that $\mathbb{E}[Z] \leq \frac{1}{1 - p} + \frac{\log n}{\log(1/p)} \rightarrow 1$. Thus, as p approaches zero the bound approaches 1. ■

(e)

Consider any particular level i and an item y that appears in that level. Derive a fairly tight upper bound (in terms of any of the relevant quantities) on the expected number of items preceding y **on level i** that are accessed when running $\text{FIND}(y)$. Also briefly describe how this bound behaves as p decreases and approaches 0.

Proof.

Let Z be the number of elements touched by $\text{FIND}(y)$ on level i that are before y .

Let Z_j be one if element j is on level i .

Let y be the r th spot in the list order.

Therefore, $Z = Z_1 + \dots + Z_{r-1}$ since we would have to touch all elements before reaching the element greater than or equal to y .

Thus, $\mathbb{E}[Z] = \sum_{j=1}^{r-1} \mathbb{E}[Z_j] p^i = (r-1)p^i \leq np^i$.

Thus, as $p \rightarrow 0$ this implies that $p^i \rightarrow 0$. Thus, $np^i \rightarrow 0$. ■

Problem 4: Spot-checking polynomials.

(a)

Recall that the *degree* of a nonzero polynomial is the largest exponent that appears in its nonzero monomials; the degree of the zero polynomial is undefined. For example, the degree of $2x^3 + 8x^7 + 1x^6$ is 7, and the degree of 376 is 0.

Also recall that every nonzero degree- d polynomial $p(x)$ has at most d (complex) *roots*, i.e., values x^* such that $p(x^*) = 0$. So, this also holds when we restrict to integer roots.

All this suggests the following *incomplete* spot-checking algorithm for zero testing:

Require: a polynomial that is either nonzero and of degree d , or is the zero polynomial

```

1: function ZEROTEST( $p(x)$ )
2:   choose an integer  $r \in \{1, 2, \dots, 2d\}$  uniformly at random
3:   if  $p(r) = 0$  then accept
4:   else reject

```

State, with justification, how to fill in the three blanks above so that:

- If $p(x)$ is the zero polynomial, then $\text{ZEROTEST}(p(x))$ accepts with certainty (i.e., with probability 1).
- If $p(x)$ is nonzero, then $\Pr[\text{ZEROTEST}(p(x)) \text{ rejects}] \geq 1/2$.

Proof.

If $p(x)$ is the zero polynomial then $p(x) = 0$ for all x . Thus, we will always accept.

If $p(x)$ is nonzero with degree d . Then we know it has at most d roots.

Thus, $\mathbb{P}[p(r) = 0] \leq \frac{d}{2d} = 1/2$. Since in the worst case we can have all d roots in our list.

Thus, $\mathbb{P}[\text{ZEROTEST}(p(x)) \text{ accepts}] = \mathbb{P}[p(r) = 0] \leq 1/2$

$\implies 1 - \mathbb{P}[\text{ZEROTEST}(p(x)) \text{ rejects}] \leq 1/2 \implies \mathbb{P}[\text{ZEROTEST}(p(x))] \geq 1/2$ ■

(b)

Accepting the zero polynomial with certainty is great, but rejecting a nonzero polynomial with probability only $1/2$ is not so great. Describe, with brief justification, *two different ways* of modifying the algorithm so that the probability of rejecting a nonzero polynomial improves to at least 99% (and the zero polynomial is still accepted with certainty).

Proof.

We could run the algorithm $\lceil \log_2 100 \rceil = 7$ times since $\frac{1}{2^7} < 0.01$, or we could increase our pool we pick from to be $\{1, \dots, 100d\}$ since $\mathbb{P}[p(r) = 0] \leq \frac{d}{100d} = 1/100$. Thus, the probability that we reject is now 99%. ■

Problem 5: Pattern-matching fingerprinting

```

1: function PATTERNMATCH( $X, Y, k$ )
2:   Pick a random prime  $p$  in the range  $\{1, \dots, k\}$ .
3:   for  $j = 1, \dots, n - m + 1$  do
4:      $X_j \leftarrow x_j \cdots x_{j+m-1}$  ▷ Substring of length  $m$  starting at  $x_j$ 
5:     if  $X_j \bmod p = Y \bmod p$  then
6:       accept
7:   reject

```

(a)

Briefly explain why if Y is a substring of X , the algorithm will always accept.

Proof.

Assume Y is a substring of X . Then for one of the loops in the for loop we will have $X_j = Y$ since we loop at all continuous length m substrings of X . Thus, for that since $X_j = Y \implies X_j \equiv Y \pmod{p}$, so we will always accept. ■

(b)

Suppose Y is not a substring of X . Derive an upper bound on the probability of a false positive in a single iteration of the algorithm. You may use the Prime Number Theorem which says the number of primes less than k is $\pi(k)$, where $\pi(k) \approx k / \ln k$.

Proof.

Assume Y is not a substring of X . Thus, for some $1 \leq j \leq n - m + 1$ we have $X_j \not\equiv Y \pmod{p}$. Thus, this implies that $p \nmid (X_j - Y)$.

Since $X_j - Y$ is an m digit binary number we can assume that m can have at most m distinct prime factors. Thus, the probability of choosing a prime factor is $\leq \frac{m \ln k}{k}$ for one loop.

Thus, for $n - m + 1$ loops we get that the probability of a false positive is

$$\leq (n - m + 1) \frac{m \ln k}{k}$$

■

(c)

In this part, you'll prove a result you need for the next part.

Let $h_j = X_j \bmod p$. Give a formula for h_j in terms of x_{j-1} (the leftmost bit in X_{j-1}), x_{j+m-1} (the bit right after X_{j-1}), and h_{j-1} .

Proof.

Consider X_j . The way to construct this from X_{j-1} is to first subtract $2^{m-1}x_{j-1}$ to get rid of the left most digit. We then left shift which is equivalent to multiplying by two.

Then we add our new rightmost digit x_{j+m-1} .

Thus, $X_j = 2(X_{j-1} - 2^{m-1}x_{j-1}) + x_{j+m-1}$. Then when we mod it by p we get.

$$h_j \equiv 2(h_{j-1} - 2^{m-1}x_{j-1}) + x_{j+m-1} \pmod{p}$$

(d)

It turns out that the current implementation of PATTERNMATCH still runs in $O(mn)$ time.

Using the result in (c), modify the algorithm so that it uses at most $O(n \log k)$ numerical operations (addition, subtraction, multiplication, modulo operation, *bit* comparison). Justify your answer.

Proof.

```

1: function PATTERNMATCH( $X, Y, k$ )
2:   Pick a random prime  $p$  in the range  $\{1, \dots, k\}$ .
3:    $h \leftarrow x_1$ 
4:   for  $i = 2, \dots, m$  do
5:      $h \leftarrow (2h + x_i) \bmod p$ 
6:    $c \leftarrow 2^{m-1} \bmod p$ 
7:    $y \leftarrow Y \bmod p$  ▷ Do it the same way as  $h$ 
8:   if  $h = y$  then accept
9:   for  $j = 2, \dots, n - m + 1$  do
10:     $h \leftarrow (2(h - c \cdot x_{j-1}) + x_{j+m-1}) \bmod p$ 
11:    if  $h = y$  then
12:      accept
13:   reject

```

To make h takes $O(m)$ operations. To make c using fast exponentiation takes $O(\log m)$.

If we do it the same way as h making y takes $O(m)$.

Now the for loop we do $O(n)$ operations then in the for loop it takes $O(\log(k))$ since we know that h is now at most size k which implies it is a $O(\log(k))$ length number. Then the comparison takes $O(\log(k))$. Thus, in total it takes $O(n \log k + m \log k) = O(n \log k)$.

The algorithm is also correct due to the proof in (c). ■

Problem 6: Diffie–Hellman.**(a)**

Suppose they use the Diffie–Hellman protocol with the small prime $p = 29$ and generator $g = 2$ for $\mathbb{Z}_p^*$. Vikram chooses secret exponent $a = 12$, and Eric chooses $b = 22$. Derive the values that they send each other and the secret key that each one computes, to conclude that they both compute the same value.

You may use a calculator for basic arithmetic (multiplication and division), but beyond that you should use the efficient algorithms that a computer would employ. Show your work (repeated squaring etc.), but you can omit the details of modular reductions. Keep numbers small by reducing modulo p where appropriate, and use negative number where they help.

Proof.

Vikram sends

$$2^{12} = 4^6 = 16^3 = 16 \cdot 256 = 16 \cdot -5 = -80 = 7 \pmod{29}$$

Eric sends

$$2^{22} = 4^{11} = 4 \cdot 16^5 = 4 \cdot 16 \cdot (-5)^2 = 4 \cdot 16 \cdot -4 = 5 \pmod{29}$$

Vikram calculates

$$5^{12} = (-4)^6 = 16^3 = 16 \cdot -5 = -80 = 7 \pmod{29}$$

Eric calculates

$$7^{22} = (-9)^{11} = -9 \cdot (-6)^5 = -9 \cdot -6 \cdot 7^2 = -9 \cdot -6 \cdot -9 = 7 \pmod{29}$$

(b)

After realizing that the parameters from the previous part are much too small (since they are breakable by hand), Vikram and Eric decide to use the Diffie–Hellman protocol with a different, huge prime p and generator g of $\mathbb{Z}_p^*$. As specified by the protocol, Vikram chooses secret $a \in \mathbb{Z}_p^*$ uniformly at random, and sends $x = g^a \bmod p$ to Eric. Similarly, Eric chooses secret $b \in \mathbb{Z}_p^*$ uniformly at random, and sends $y = g^b \bmod p$ to Vikram.

Unbeknownst to them, Lambda is in the middle of their network, and is able to intercept *and undetectably replace* what they send over the network with other values of his choice. (In class we considered only an *eavesdropper* Eve who can merely read all the network traffic; this Lambda is more powerful.)

Specifically, Lambda chooses a secret $c \in \mathbb{Z}_p^*$ himself, and sends $z = g^c \bmod p$ to both Vikram and Eric in place of their messages to each other. That is, Vikram thinks that Lambda's message came from Eric, and Eric thinks that Lambda's message came from Vikram.

Give an expression for the key that Vikram K_V computes, and the key K_E that Eric computes, in terms of the secret exponents a, b, c and the public values p, g .

Proof.

$$K_V = g^{ac} \pmod{p} \text{ and } K_E = g^{bc} \pmod{p}$$

(c)

Vikram and Eric work on the homework solutions by encrypting their messages using the keys they computed in the previous part (which they believe are the same key). Lambda knows the encryption scheme that Vikram and Eric are using. Describe how Lambda can decrypt all of Vikram and Eric's communications without being detected, i.e., so that they believe they are communicating confidentially, and when they decrypt the ciphertexts they receive, they get exactly the messages that were intended for them.

A valid solution should show the following:

- how Lambda can successfully decrypt all of the ciphertexts that Vikram and Eric send to each other;
- how Lambda can avoid detection by replacing the ciphertexts that Vikram and Eric send with ones that will yield the intended messages when they decrypt (using what they believe to be their shared key).

Proof.

First Vikram sends a message and encrypts it with K_v . Then Lambda decrypts it with K_v then encrypts it with K_E and sends it to Eric. Then Eric decrypts it with K_E thinking this is the actual Key.

If Eric sends a message do the same in reverse. ■

Problem 7:**(a)***Proof.*

Let $L \in \text{BPP}$, then there is a probabilistic polynomial time turing machine M and a constant $1/3$ s.t.

$$x \in L \quad \mathbb{P}[M(x) \text{ accepts}] \geq 2/3$$

$$x \notin L \quad \mathbb{P}[M(x) \text{ accepts}] \leq 1/3$$

Fix $\epsilon \in (0, 1/2)$

Define $M'(x)$ s.t. it runs $M(x)$ k times (we will define later), and $M'(x)$ accepts if $M(x)$ accepts more than $k/2$ times and reject otherwise.

This is polynomial time since M is polynomial time.

Let S be the number of times that $M(x)$ accepts, and let X_i be one if $M(x)$ accepts on the i th loop. Thus, $S = X_1 + X_2 + \dots + X_k$.

If $x \in L$, each loop accepts with probability at least $2/3$. So $E[S] \geq 2k/3$ and $E[S/k] \geq 2/3$

Thus,

$$\mathbb{P}[S/k \leq E[S/k] - 1/6] \leq \mathbb{P}[S \leq k/2] \leq e^{-k/18}$$

If $x \notin L$, each loop accepts with probability at most $1/3$. So $E[S] \leq k/3$ and $E[S/k] \leq 1/3$

$$\mathbb{P}[S/k \geq E[S/k] + 1/6] \leq \mathbb{P}[S \geq k/2] \leq e^{-k/18}$$

Thus, the error is at most $e^{-k/18}$, so for $e^{-k/18} \leq \epsilon$ pick $k \geq 18 \ln(1/\epsilon)$.

Thus, we have made a polynomial time function with error at most ϵ . ■

(b)*Proof.*

If $L \in \text{RP}$ via a randomized machine M that runs in polynomial time and accepts with probability at least $1/2$ when $x \in L$ (and never accepts if $x \notin L$), we simply take the certificate C to be the particular sequence of coin-flip outcomes that makes M accept. The verifier $V(x, C)$ deterministically simulates M on input x using randomness C and

accepts exactly if M does. If $x \in L$, there must exist some C for which V accepts; if $x \notin L$, no C can cause M (and hence V) to accept. Therefore L has a polynomial-time verifier with certificate C and lies in NP. ■

(c)

Proof.

Assume $\text{NP} \subseteq \text{BPP}$. This implies that $\text{SAT} \in \text{BPP}$. Thus, there is a probabilistic turing machine D that runs in polynomial time s.t.

$$\phi \in \text{SAT} \quad \mathbb{P}[D(\phi) \text{ accepts}] \geq 2/3$$

$$\phi \notin \text{SAT} \quad \mathbb{P}[D(\phi) \text{ accepts}] \leq 1/3$$

Shown in (a) we can build a polynomial time probabilistic turning machine D that has error $\leq \epsilon$. Pick $\epsilon = \frac{1}{2(n+1)}$. where n is the number of variable's of ϕ .

Let ϕ have literals $x_1, \dots, x_n$.

Now construct a new machine A s.t.

If $D'(\phi)$ reject's then reject.

Then for $i \in \{1, \dots, n\}$

First set $x_i = \text{true}$, then run $D(\phi(s_1, s_2, \dots, s_{i-1}, \text{true}, \dots, x_n))$ where s_j are already defined from the previous iterations.

If $D(\phi(s_1, s_2, \dots, s_{i-1}, \text{true}, \dots, x_n))$ accepts else

$x_i = \text{false}$

Now finally after all the loops we will have fixed all x_i 's call this assignment a .

If $\phi(a)$ is true. Then return accept else reject.

Runtime:

This runs in polynomial time since we run D at most linear times, and everything else is polynomial.

Probability:

First off there is no errors if $\phi \notin \text{SAT}$ since we only accept if we have found an assignment for ϕ and reject if our assignment we made does not work.

Thus, $\mathbb{P}[D'(\text{SAT}) \text{ accepts}] = 0$

Then for if $\phi \in \text{SAT}$ then

We call D at most $(n+1)$ times, so for each we know that each have an error of less than ϵ . Thus, the chance all the calls are correct is $1 - (n+1)\epsilon = 1 - \frac{n+1}{2n+2} \geq 1/2$.

Thus, for $\mathbb{P}[D'(\phi) \text{ accepts}] = \mathbb{P}[D(\phi) \text{ accepts all of the times}] \leq 1/2$

Therefore, $\text{SAT} \in \text{RP}$, and since SAT is NP-complete, this implies that $\text{NP} \subseteq \text{RP}$. ■